

Witness Generation for JIT Compiler Fault Localization

Robby Votta, Dr. HeuiChan (Terrence) Lim

Department of Mathematics and Computer Science, Davidson College

Introduction

Background

- Just-in-time (JIT) compilers, (e.g. Google V8) optimize frequently executed code at runtime
- Optimization bugs are difficult to localize due to their dynamic execution and architectural complexity [4]

Prior Work

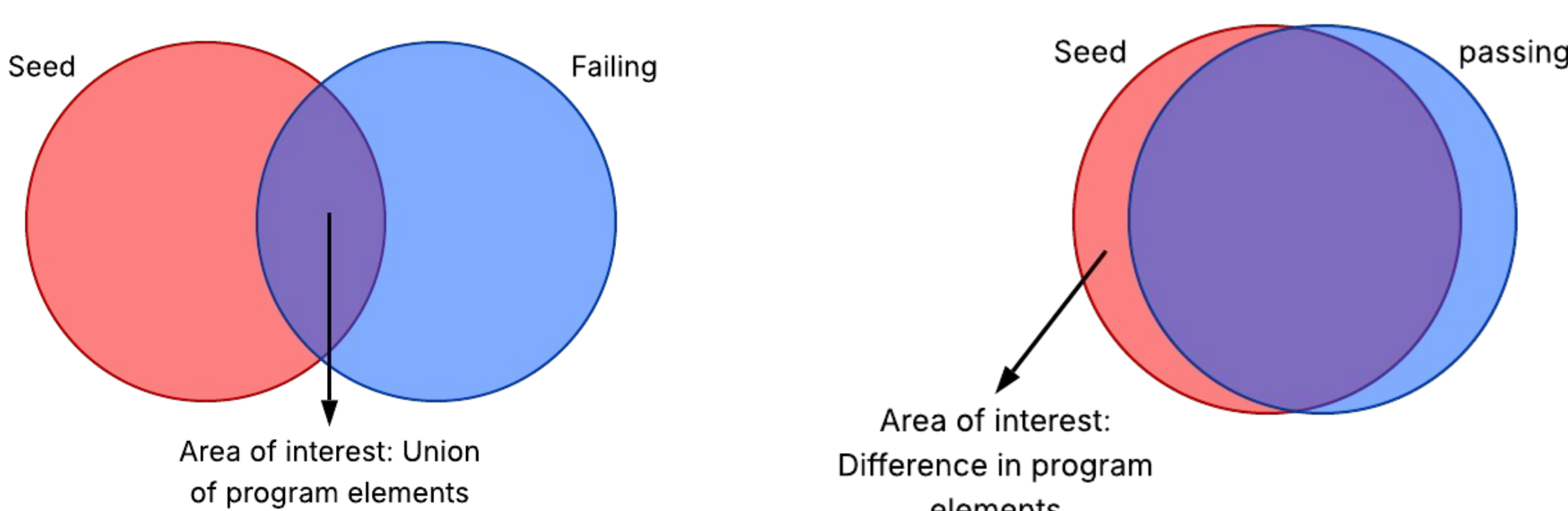
- generate passing witness programs, collect compiler coverage information, rank source files by suspiciousness [2]
- this method reviews non-JIT components [4], and code must run repeatedly to reach the optimizing tier

Proposal

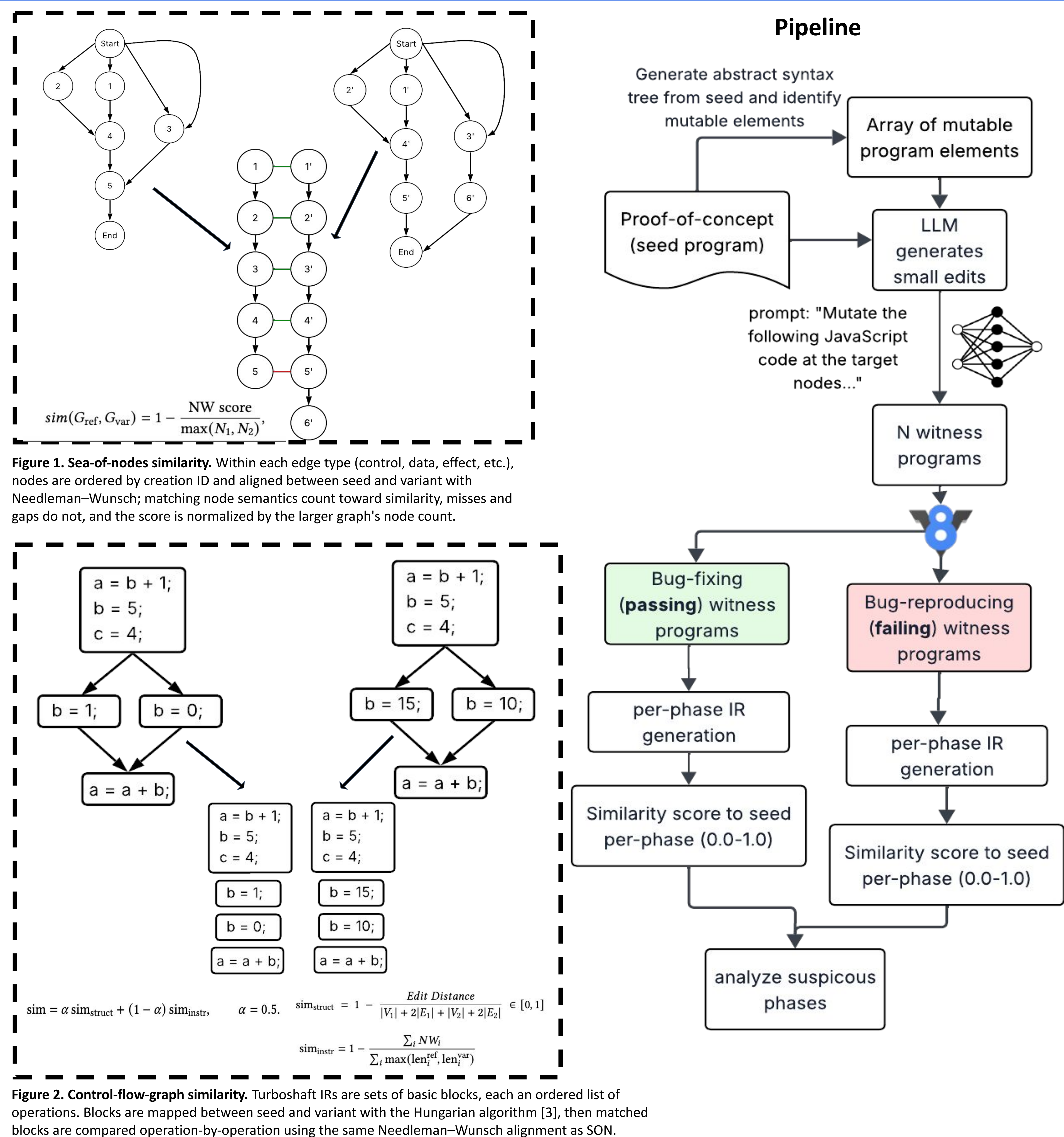
- Compare the compiler's intermediate representation (IR), phase by phase, between a proof-of-concept (PoC) and LLM-generated witnesses programs localizing to the faulty optimization phase

Objectives

- Generate passing witness programs highly similar to the PoC and failing witness programs that are less similar to the PoC
- Create a valid similarity metric for both sea-of-node (SON) and control-flow (CFG) IRs.
- Correctly localize PoC to its buggy optimization phase



Methods



Results



Figure 3: Ranking most divergent phases by how often they appear among each passing witness program's most divergent phases. Highlighted bars mark the known buggy phase. In the case of the rotate-right bug (orange), the actual buggy phase was ranked number 10. The first three are TurboFan bugs, represented as sea-of-nodes (SON) graphs. The last is a Turboshift bug, represented as a control-flow graph (CFG).

Figure 4: Per-phase Area Under the ROC Curve (AUC). Calculate the probability that a randomly chosen failing variant is more similar to the seed than a randomly chosen passing one. The highest-AUC phase marks where bug-fixing variants diverge from the seed while bug-reproducing variants remain similar.

Bug	Known Buggy Phase	Highest AUC Phase	AUC
40056729	MachineOperator	TFStoreStoreElimination	0.939
40094915	TFSimplifiedLowering	TFTyper	0.800
40052054	TFTyper	TFtypedLowering	0.961
491881374	TurboshiftLoopUnrolling	TurboshiftLoopUnrolling	1.000

Future Work

- Continue developing the LLM-driven witness generation with an iterative feedback-driven localization based on the first batch of PoC variants
- Current output points to a buggy optimization phase. We want to localize to the file-level and function-level of the V8 C++ source code.
- The implementation is only tested on 4 real-world V8 bugs. We plan to further analysis to 10-15 more bugs

Acknowledgments

We thank the Davidson Research Initiative grant for funding and supporting this research. We would also like to thank the Developers at Google V8 providing feedback towards our implementation throughout development.

References

- Q. I. Sarhan and Á. Beszédés, "A Survey of Challenges in Spectrum-Based Software Fault Localization," IEEE Access, vol. 10, pp. 10618–10639, 2022, doi: 10.1109/ACCESS.2022.3144079.
- P. P. F. Chan and C. Collberg, "A Method to Evaluate CFG Comparison Algorithms," in Proc. 14th Int. Conf. Quality Software (QSI), Dallas, TX, USA, 2014, pp. 95–104, doi: 10.1109/QSI.2014.28.
- Institute for Clinical and Translational Research, Albert Einstein College of Medicine, "What is the Area under the Receiver Operating Curve (AUC) in Prediction?" [Online]. Available: <https://einsteinmed.edu/uploadedfiles/centers/ict/new/area-under-roc-curve.pdf>
- HeuiChan Lim and Saumya Debray, 2021. Automated bug localization in JIT compilers. In Proceedings of the 17th ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE 2021). Association for Computing Machinery, New York, NY, USA, 153–164. <https://doi.org/10.1145/3453933.3454021>